RIVIERA MED — MASTER BUILD SPECIFICATION FOR AI CODING ASSISTANT

Complete Production-Ready Website · Next.js 14 · TypeScript · Senior-First Healthcare UX

> Paste this entire document into Cursor, Windsurf, v0, or Bolt.new
> Version 1.0 · April 2026 · info@riviera-med.com

EXECUTIVE SUMMARY

You are building a **complete production website** for Riviera Med GmbH, a private Spitex (home healthcare) service in Thun, Switzerland. The site serves **two primary audiences**:

1. **Public (conversion-focused):** Elderly clients (65+) and their adult children (45–65) seeking home care services. They need to book an initial consultation (Erstgespräch) and understand transparent pricing.
2. **Staff (internal tools):** Riviera Med employees need a secure admin dashboard to manage inquiries, clients, schedules, and reports.

Critical context: This is a **healthcare website for seniors**. Every design, interaction, and UX pattern must prioritize:

- **Senior-first accessibility** (18px+ text, 48px+ touch targets, high contrast, one action per screen)
- **Swiss German audience** (de-CH locale, Sie-form throughout, no "ß")
- **Trust & transparency** (healthcare is high-stakes — no manipulative dark patterns, honest pricing, phone fallback always visible)

Business objective: Convert website visitors into qualified leads by collecting structured intake data with minimal cognitive load, while providing transparent cost information upfront.

TECH STACK (non-negotiable)

```
Framework:      Next.js 14.2+ (App Router, Server Components, Server Actions)
Language:       TypeScript 5.4+ (strict mode)
Package Mgr:    pnpm 9+
Styling:        Tailwind CSS 3.4 + CSS Modules for design tokens
Forms:          React Hook Form 7.5 + @hookform/resolvers + Zod 3.22
Data fetching:  TanStack Query (React Query) v5
State:          Zustand 4+ (global) + useState (local)
Auth:           Auth.js (NextAuth) v5 with Credentials + JWT strategy
```

```
Database:       PostgreSQL 16 via Supabase (or Neon, Vercel Postgres)
ORM:            Drizzle ORM 0.30+
Email:          Resend (transactional) + React Email templates
Charts:         Recharts 2.12
Icons:          Lucide React 0.383
Animation:      Motion (framer-motion successor) — minimal, honors prefers-
reduced-motion
Validation:     Zod (shared client/server schemas)
Testing:        Vitest (unit) + Playwright (E2E)
Deployment:     Vercel (Edge Runtime where possible)
Analytics:      PostHog (events) + Vercel Analytics (Web Vitals)
Error tracking: Sentry
```

Environment variables required:

bash

```bash
# .env.local
DATABASE_URL=postgresql://...
NEXTAUTH_SECRET=...
NEXTAUTH_URL=http://localhost:3000
RESEND_API_KEY=re_...
POSTHOG_KEY=phc_...
SENTRY_DSN=https://...
```

DESIGN SYSTEM TOKENS

All tokens are defined in **three formats** (use the JSON as source of truth):

Color Palette

typescript

```
// Primary — Deep Pine (brand color)
const primary = {
  50:  '#F0F5F4', 100: '#DCE8E6', 200: '#B8D1CD', 300: '#8AB1AB',
  400: '#5C8E87', 500: '#3F6F68', 600: '#2C5651', 700: '#1F4340', // ← default brand
  800: '#163331', 900: '#0E2422', 950: '#071512',
}

// Secondary — Warm Sand (accent, rare use)
const secondary = {
  50:  '#FBF8F2', 100: '#F5EDDC', 200: '#EADCBA', 300: '#DBC48A',
  400: '#C7A85C', // ← accent baseline
  500: '#A8893F', 600: '#856B30', 700: '#5F4C22',
}

// Neutral — Slate (workhorse)
const neutral = {
  0:   '#FFFFFF', 50:  '#FAFAF9', // ← page background
  100: '#F4F4F2', 200: '#E7E7E4', // ← default border
  300: '#D1D1CD', 400: '#A8A8A2', 500: '#787873',
  600: '#52524E', 700: '#3D3D3A', // ← body text
  800: '#272725', // ← headings
  900: '#161614', 950: '#0A0A09',
}

// Semantic
const semantic = {
  success: { 50: '#EDF6F0', 500: '#2D7A4F', 700: '#1F5638' },
  warning: { 50: '#FAF4E5', 500: '#B5841F', 700: '#825D11' },
  error:   { 50: '#F9ECEB', 500: '#A8302A', 700: '#7A211C' },
  info:    { 50: '#EBF1F6', 500: '#2C5C7A', 700: '#1F4257' },
}
```

Typography (Switzer font family)

typescript

```
// Font sizes (9-step scale, ratio 1.25)
const fontSize = {
  xs:   '12px', sm:   '14px',
  base: '18px', // ← body baseline (NOT 16px — seniors need larger)
  lg:   '20px', xl:   '24px',
  '2xl': '32px', '3xl': '40px',
  '4xl': '56px', '5xl': '72px', '6xl': '96px',
}

// Font weights
const fontWeight = { regular: 400, medium: 500, semibold: 600 }

// Line heights
const lineHeight = {
  none: 1.0, tight: 1.1, snug: 1.2, normal: 1.3,
  relaxed: 1.5, loose: 1.6, // ← body default
}
```

Spacing (8px grid)

typescript

```
const spacing = {
  0: '0', 1: '4px', 2: '8px', 3: '12px', 4: '16px', 5: '20px',
  6: '24px', 8: '32px', 10: '40px', 12: '48px', 16: '64px',
  20: '80px', 24: '96px', 32: '128px', 40: '160px', 48: '192px',
}
```

Border Radius (minimal & modern)

typescript

```
const radius = {
  none: '0', xs: '2px', sm: '4px', md: '8px', full: '9999px',
}
```

Import full design system from: `/design-system/tokens.json` and `/design-system/tokens.css` (provided separately)

SITE STRUCTURE & ROUTES

```
app/
├── (public)/                    ← Public-facing site
│   ├── page.tsx                 → / (Homepage)
```

```
│   ├── leistungen/
│   │   ├── page.tsx              → /leistungen/ (Services hub)
│   │   ├── pflege-betreuung/page.tsx
│   │   ├── hauswirtschaft/page.tsx
│   │   ├── nachtwachen/page.tsx
│   │   └── physiotherapie/page.tsx
│   ├── tarife/page.tsx           → /tarife/ (Pricing)
│   ├── ueber-uns/page.tsx        → /ueber-uns/ (About)
│   ├── partner/page.tsx          → /partner/ (Partners)
│   ├── kontakt/page.tsx          → /kontakt/ (Contact + Inquiry Form)
│   ├── datenschutz/page.tsx
│   └── impressum/page.tsx
│
├── (admin)/                      ← Staff dashboard (auth required)
│   ├── login/page.tsx
│   ├── forgot/page.tsx
│   ├── reset/page.tsx
│   └── dashboard/
│       ├── page.tsx              → /dashboard/ (Overview)
│       ├── clients/page.tsx
│       ├── inquiries/page.tsx
│       ├── schedule/page.tsx
│       └── reports/page.tsx
│
├── api/
│   ├── auth/[...nextauth]/route.ts
│   ├── inquiry/route.ts          → POST /api/inquiry (form submission)
│   ├── pricing/email/route.ts → POST /api/pricing/email (calculator summary)
│   ├── staff/
│   │   ├── clients/route.ts      → GET /api/staff/clients (faceted search)
│   │   ├── inquiries/route.ts
│   │   └── dashboard/route.ts
│   └── health/route.ts           → GET /api/health (uptime check)
│
├── layout.tsx                    ← Root layout
├── globals.css                   ← Import tokens.css + Tailwind
└── providers.tsx                 ← QueryClient, Auth, Toaster
```

PAGE-SPECIFIC REQUIREMENTS

1. Homepage (/)

Hero:

- Eyebrow: "Private Spitex · Region Thun"

- Headline: "Pflege zu Hause. Mit Herz." (H1, 56–72px)
- Subheadline: "Bedarfsgerechte Pflege und Betreuung – kompetent, herzlich, rund um die Uhr in der Region Thun."
- CTA primary: "Erstgespräch vereinbaren →" (links to /kontakt/)
- CTA secondary: "Tarife ansehen" (links to /tarife/)
- Trust strip: " ✓ Alle Krankenkassen anerkannt · ✓ KLV-konform · ✓ 24/7 · 365 Tage · ✓ Region Thun & Berner Oberland"

Services section: 3-column grid of ServiceCards (Pflege, Betreuung, Hauswirtschaft, Nachtwachen, Physiotherapie)

Testimonials: 3-card carousel or grid with:

```tsx
<TestimonialCard
  quote="Mein Vater wollte um keinen Preis ins Heim. Riviera Med hat es möglich gema
  author="Sandra M."
  role="Tochter"
  location="Hilterfingen"
/>
```

Partner logos: Greyscale horizontal strip (Spital Thun, GSI, Pro Senectute, etc.)

FAQ section: Accordion with 8 questions (see copy document)

CTA banner: "Sind wir die Richtigen für Ihre Familie?" + CTA

Footer: 4-column (Brand, Leistungen, Unternehmen, Erreichbarkeit) + legal links

2. Contact Page (`/kontakt/`) — THE CONVERSION PAGE

Two-column layout:

- **Left:** Multi-step inquiry form wizard (see Module 1 below)
- **Right:** Direct contact block with phone `079 237 55 21`, address, WhatsApp button

Form must implement:

- 7-step wizard (one question per screen for seniors)
- Draft persistence in localStorage (key: `rm_inquiry_draft_v1`, 24h TTL)
- Zod validation per step
- Resume detection on mount
- Review step before submit
- Success confirmation with phone fallback

API endpoint: `POST /api/inquiry`

- Stores submission in DB
- Sends email to `info@riviera-med.com` (Resend)
- Auto-reply to user email
- Returns 201 on success

3. Pricing Page (`/tarife/`)

Sticky anchor nav: Pflegeleistungen | Hauswirtschaft | Nachtwachen | Finanzierung

Interactive pricing calculator (see Module 2):

- Service toggles (Pflege, Hauswirtschaft, Nachtwache, Physio)
- Hours/week sliders
- Days/week slider
- Age toggle (≤65 / 65+)
- **Live computed results:**
  - Krankenkasse-covered amount
  - Patientenbeteiligung (max CHF 15.35/day)
  - Self-pay total
  - Monthly estimate range (low/expected/high)

Tariff tables:

```tsx
<TariffTable>
  <tr><td>Hauswirtschaft</td><td>pro Stunde</td><td>CHF 55.00</td></tr>
  <tr><td>Wegpauschale</td><td>pro Besuch</td><td>CHF 7.00</td></tr>
  <tr><td>Präsenz-Nachtwache</td><td>8h Nacht</td><td>CHF 400.00</td></tr>
  {/* ... */}
</TariffTable>
```

Financing accordion: Krankenkasse, Kanton Bern (GSI), Patientenbeteiligung, Ergänzungsleistungen

PDF download: Tarifblatt.pdf link

4. About Page (`/ueber-uns/`)

Team section: 3-column grid with:

```tsx
```

```
<TeamCard
  photo="/team/muad.jpg"
  name="Muad Amiin"
  role="Geschäftsführer (CEO)"
  bio="Mit langjähriger Erfahrung in der Pflege..."
/>
```

Service area map: Google Maps embed showing Thun, Spiez, Hünibach, etc.

Stats counters: 15+ Jahre · 24/7 · 5 Gemeinden · 100% Krankenkassen

5. Admin Dashboard (`/dashboard/`)

Protected route (requires auth via middleware)

Layout:

- Sidebar nav
- Main content area with 4 KPI cards + 4 widget areas

Widgets (TanStack Query hooks):

- `useTodaysVisits()` → list of today's scheduled visits
- `useNewInquiries()` → list of new inquiry form submissions
- `useActiveClients()` → count + quick access
- `useWeeklyHours()` → chart (Recharts bar)

Real-time: Server-Sent Events for new inquiry notifications

INTERACTIVE MODULES (5 core features)

MODULE 1: Multi-Step Inquiry Form

Location: `/kontakt/` page, left column

Flow:

1. "Worum geht es?" (service type — radio cards)
2. "Für wen?" (recipient — radio cards)
3. "Wann benötigt?" (urgency — radio cards)
4. Contact info (name*, phone*, email)
5. "Wann erreichen wir Sie?" (best time — radio)
6. Optional message (textarea)
7. Review + Datenschutz checkbox

8. Success confirmation

State management:

tsx

```tsx
// hooks/useInquiryWizard.ts
import { useReducer } from 'react'

type State = {
  formData: Partial<InquiryFormData>
  currentStep: number  // 1-7
  status: 'idle' | 'step_view' | 'submitting' | 'success' | 'api_error'
  errors: Partial<Record<keyof InquiryFormData, string>>
}

type Action =
  | { type: 'UPDATE'; field: keyof InquiryFormData; value: any }
  | { type: 'NEXT' }  // validates current step, advances if ok
  | { type: 'BACK' }
  | { type: 'SUBMIT_START' }
  | { type: 'SUBMIT_SUCCESS' }
  | { type: 'SUBMIT_FAIL'; message: string }

function reducer(state: State, action: Action): State { /* ... */ }
```

Validation schema (Zod):

tsx

```
// shared/schemas/inquiry.ts
import { z } from 'zod'

export const inquirySchema = z.object({
  serviceType: z.enum(['pflege', 'hauswirtschaft', 'nachtwache', 'physio', 'multiple
  recipient: z.enum(['self', 'partner', 'parent', 'other']),
  urgency: z.enum(['immediate', 'within_week', 'within_month', 'later']),
  name: z.string().trim().min(2, 'Bitte geben Sie Ihren Namen ein.'),
  phone: z.string().trim().regex(
    /^(\+41|0)[\s-]?[1-9](?:[\s-]?\d){8}$/,
    'Bitte geben Sie eine gültige Schweizer Telefonnummer ein.'
  ),
  email: z.string().trim().email().optional().or(z.literal('')),
  bestTime: z.enum(['morning', 'afternoon', 'evening', 'any']),
  message: z.string().max(2000).optional(),
  consent: z.literal(true, {
    errorMap: () => ({ message: 'Bitte bestätigen Sie die Datenschutzerklärung.' }),
  }),
})

export type InquiryFormData = z.infer<typeof inquirySchema>
```

Draft persistence:

```tsx
```

```ts
// hooks/useDraftPersistence.ts
useEffect(() => {
  const draft = localStorage.getItem('rm_inquiry_draft_v1')
  if (draft) {
    const { data, savedAt } = JSON.parse(draft)
    if (Date.now() - savedAt < 24 * 60 * 60 * 1000) {
      // Show resume dialog
    }
  }
}, [])

useEffect(() => {
  // Auto-save on changes (debounced 400ms)
  const timer = setTimeout(() => {
    localStorage.setItem('rm_inquiry_draft_v1', JSON.stringify({
      data: state.formData,
      savedAt: Date.now(),
    }))
  }, 400)
  return () => clearTimeout(timer)
}, [state.formData])
```

Component tree:

```
src/features/inquiry-wizard/
├── InquiryWizard.tsx
├── components/
│   ├── WizardShell.tsx
│   ├── ProgressIndicator.tsx
│   ├── StepNavigator.tsx
│   └── ResumeDialog.tsx
├── steps/
│   ├── Step1Service.tsx
│   ├── Step2Recipient.tsx
│   ├── Step3Urgency.tsx
│   ├── Step4Contact.tsx
│   ├── Step5BestTime.tsx
│   ├── Step6Message.tsx
│   ├── Step7Review.tsx
│   └── Step8Success.tsx
└── hooks/
    ├── useInquiryWizard.ts
    └── useDraftPersistence.ts
```

MODULE 2: Real-Time Pricing Calculator

Location: `/tarife/` page, embedded section

Inputs (controlled state):

- Service checkboxes + hours sliders (Pflege, Hauswirtschaft, Nachtwache, Physio)
- Days per week slider (1–7)
- Age toggle (≤65 / 65+)

Computation (pure function):

tsx

```tsx
// features/pricing-calculator/lib/calculate.ts

type CalculatorInputs = {
  services: {
    pflege: { enabled: boolean; hoursPerWeek: number }
    hauswirtschaft: { enabled: boolean; hoursPerWeek: number }
    nachtwache: { enabled: boolean; nightsPerWeek: number; tier: 'praesenz' | 'sitz'
    physio: { enabled: boolean; hoursPerWeek: number }
  }
  daysPerWeek: number
  age65plus: boolean
}

const CHF = {
  hauswirtschaftPerHour: 55,
  wegpauschalePerVisit: 7,
  nachtPraesenz: 400,
  nachtSitz: 420,
  nachtDiplomiert: 460,
  patientShareMaxPerDay: 15.35,
  pflegeReferencePerHour: 76, // KLV estimate
}

const WEEKS_PER_MONTH = 4.33

export function calculate(inputs: CalculatorInputs): CalculatorResults {
  // Compute krankenkasseCovered, selfPay, patientShare, wegpauschale
  // Return { totalLow, totalExpected, totalHigh, breakdown }
}
```

Live results panel:

tsx

```
<ResultsPanel>
  <ResultRow label="Krankenkasse übernimmt" amount={results.krankenkasseCovered} cov
  <ResultRow label="Patientenbeteiligung" amount={results.patientShare} />
  <ResultRow label="Selbstzahlung" amount={results.selfPay} />
  <Divider />
  <ResultTotal
    low={results.totalLow}
    expected={results.totalExpected}
    high={results.totalHigh}
  />
</ResultsPanel>
```

Optional: "Per E-Mail erhalten" button → opens modal → collects email → sends summary via `POST /api/pricing/email`

MODULE 3: Faceted Search (Staff Client Database)

Location: `/dashboard/clients/` (protected route)

Search surface:

- Free-text query (name, address, phone) — debounced 300ms
- Facets: Status, Service type, Region, Primary caregiver, Last visit date
- Sort: Name, Last visit, Next scheduled
- Pagination: server-side, 25 per page

URL-synced state:

tsx

```tsx
// hooks/useClientSearch.ts
import { useUrlState } from './useUrlState'
import { useQuery } from '@tanstack/react-query'

export function useClientSearch() {
  const [params, setParams] = useUrlState({
    q: '',
    status: [] as string[],
    region: [] as string[],
    sort: 'name_asc',
    page: 1,
  })

  const debouncedQ = useDebounce(params.q, 300)

  const query = useQuery({
    queryKey: ['clients', { ...params, q: debouncedQ }],
    queryFn: ({ signal }) => searchClients({ ...params, q: debouncedQ }, { signal }),
    placeholderData: keepPreviousData,
  })

  return { params, setParams, results: query.data, isLoading, error }
}
```

Component tree:

```
src/features/client-search/
├── ClientSearchPage.tsx
├── components/
│   ├── SearchToolbar.tsx
│   ├── FacetSidebar.tsx
│   ├── ResultsTable.tsx
│   ├── Pagination.tsx
│   └── ActiveFiltersBar.tsx
└── hooks/
    ├── useClientSearch.ts
    └── useUrlState.ts
```

MODULE 4: Staff Dashboard

Location: `/dashboard/` (protected route)

Layout: 4 KPI cards + 4 widgets

Data fetching pattern (Server Component prefetch + Client hydration):

tsx

```tsx
// app/(admin)/dashboard/page.tsx
import { dehydrate, HydrationBoundary } from '@tanstack/react-query'
import { getQueryClient } from '@/lib/queryClient'

export default async function DashboardPage() {
  const qc = getQueryClient()

  await Promise.all([
    qc.prefetchQuery({ queryKey: ['visits', 'today'], queryFn: fetchTodaysVisits }),
    qc.prefetchQuery({ queryKey: ['inquiries', 'new'], queryFn: fetchNewInquiries })
  ])

  return (
    <HydrationBoundary state={dehydrate(qc)}>
      <DashboardShell>
        <KpiRow />
        <TodaysVisitsWidget />
        <NewInquiriesWidget />
        <HoursByServiceChart />
      </DashboardShell>
    </HydrationBoundary>
  )
}
```

Widget pattern (per-widget error boundaries):

tsx

```tsx
// widgets/TodaysVisitsWidget.tsx
export function TodaysVisitsWidget() {
  const { data, isLoading, error, refetch } = useQuery({
    queryKey: ['visits', 'today'],
    queryFn: fetchTodaysVisits,
  })

  if (isLoading) return <WidgetSkeleton />
  if (error) return <WidgetError onRetry={refetch} />
  if (!data || data.length === 0) return <WidgetEmpty message="Heute keine Einsätze

  return (
    <div className="rounded-md border p-8">
      <h3>Heutige Einsätze</h3>
      {data.map(visit => <VisitRow key={visit.id} visit={visit} />)}
    </div>
  )
}
```

Charts (Recharts):

```tsx
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts'

<ResponsiveContainer width="100%" height={280}>
  <BarChart data={data}>
    <XAxis dataKey="service" stroke="var(--color-neutral-500)" />
    <YAxis stroke="var(--color-neutral-500)" />
    <Tooltip />
    <Bar dataKey="hours" fill="var(--color-primary-700)" radius={[2, 2, 0, 0]} />
  </BarChart>
</ResponsiveContainer>
```

MODULE 5: Authentication Lifecycle

Location: `/login/`, `/forgot/`, `/reset/`

Auth strategy: NextAuth v5 with Credentials provider + JWT

Auth config:

```tsx
```

```
// server/auth.config.ts
import NextAuth from 'next-auth'
import Credentials from 'next-auth/providers/credentials'
import { verifyPassword } from '@/lib/crypto'

export const { auth, signIn, signOut } = NextAuth({
  providers: [
    Credentials({
      credentials: {
        email: { label: "E-Mail", type: "email" },
        password: { label: "Passwort", type: "password" },
      },
      async authorize(credentials) {
        const user = await db.query.users.findFirst({
          where: eq(users.email, credentials.email),
        })
        if (!user) return null

        const valid = await verifyPassword(credentials.password, user.passwordHash)
        if (!valid) return null

        return { id: user.id, email: user.email, name: user.name }
      },
    }),
  ],
  session: { strategy: 'jwt', maxAge: 30 * 60 }, // 30 min
  pages: {
    signIn: '/login',
  },
})
```

Middleware (route protection):

```
tsx
```

```
// middleware.ts
import { auth } from '@/server/auth.config'
import { NextResponse } from 'next/server'

export default auth((req) => {
  const isLoggedIn = !!req.auth
  const isAdminRoute = req.nextUrl.pathname.startsWith('/dashboard')

  if (isAdminRoute && !isLoggedIn) {
    return NextResponse.redirect(new URL('/login', req.url))
  }

  return NextResponse.next()
})

export const config = {
  matcher: ['/dashboard/:path*', '/login', '/forgot', '/reset'],
}
```

Login form:

tsx

```tsx
// features/auth/LoginForm.tsx
'use client'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { signIn } from 'next-auth/react'

const schema = z.object({
  email: z.string().email(),
  password: z.string().min(1),
})

export function LoginForm() {
  const { register, handleSubmit, formState: { errors } } = useForm({
    resolver: zodResolver(schema),
  })
  const [isSubmitting, setIsSubmitting] = useState(false)

  const onSubmit = handleSubmit(async ({ email, password }) => {
    setIsSubmitting(true)
    const result = await signIn('credentials', {
      email,
      password,
      redirect: false,
    })
    if (result?.error) {
      // Show error
    } else {
      router.push('/dashboard')
    }
    setIsSubmitting(false)
  })

  return (
    <form onSubmit={onSubmit}>
      <Field label="E-Mail" error={errors.email?.message}>
        <input type="email" {...register('email')} autoComplete="username" />
      </Field>
      <Field label="Passwort" error={errors.password?.message}>
        <input type="password" {...register('password')} autoComplete="current-passw
      </Field>
      <Button type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'Wird angemeldet...' : 'Anmelden'}
      </Button>
    </form>
```

```
  )
}
```

Security requirements:

- Password hashing: Argon2id via `@node-rs/argon2`
- Rate limiting: 5 attempts per 15 min per IP+email
- Session: 30 min idle timeout with 1-min warning banner
- CSRF: double-submit cookie (NextAuth built-in)
- Email enumeration prevention: identical response for exists/doesn't exist

---

CONTENT & COPY

All German copy is provided in a separate document (`riviera-med-website-copy.md`). Key highlights:

Brand tone: Professional & Authentic

- Use Sie-form throughout (never du)
- No "ß" (Swiss German uses "ss")
- No manipulative urgency ("Nur noch 3 Plätze!")
- Always show phone fallback: **079 237 55 21**

Homepage hero:

```tsx
<h1>Pflege zu Hause. Mit Herz.</h1>
<p>Bedarfsgerechte Pflege und Betreuung – kompetent, herzlich, rund um die Uhr in de
```

Button labels:

- Primary CTA: "Erstgespräch vereinbaren →"
- Secondary: "Tarife ansehen" / "Mehr erfahren →"
- Phone: "☎ 079 237 55 21"
- WhatsApp: "Per WhatsApp schreiben"

Error messages (examples):

- Name field: "Bitte geben Sie Ihren Namen ein."
- Phone field: "Bitte geben Sie eine gültige Schweizer Telefonnummer ein."
- Email field: "Bitte geben Sie eine gültige E-Mail-Adresse ein."

- Consent checkbox: "Bitte bestätigen Sie die Datenschutzerklärung."

Form success message:

```
✓ Vielen Dank für Ihre Anfrage, [Name].

Wir melden uns innerhalb von 24 Stunden bei Ihnen – meist schneller.
Bei dringenden Anliegen rufen Sie uns gerne direkt an: 079 237 55 21
```

CRITICAL CONSTRAINTS (must-haves)

Senior-First Accessibility

1. **18px minimum body text** (not 16px — already in design system)
2. **48px minimum touch targets** for all interactive elements
3. **No hover-only interactions** — every hover state has focus/tap equivalent
4. **High contrast** — WCAG AAA for body text (neutral-700 on neutral-50 = 11.8:1)
5. **One primary action per screen** for complex flows (multi-step form)
6. **Forgiving validation:**
   - Auto-format phone numbers (accept `0792375521`, `079 237 55 21`, `+41792375521`)
   - Trim whitespace silently
   - No auto-submit on autofill
7. **Draft persistence** — localStorage with 24h TTL on inquiry form
8. **Always-visible escape hatch** — phone number in header: "Hilfe? 079 237 55 21"
9. **No time pressure** — no countdown timers, no auto-logout on public pages
10. **Predictable behavior** — no surprise modals, no auto-advance
11. **Honor** `prefers-reduced-motion` — already in design tokens CSS
12. **Confirm destructive actions** — "Wirklich abbrechen?" before discarding form data

Performance

- **LCP ⩽ 1.5s** (homepage hero image)
- **INP ⩽ 200ms** (form interactions)
- **CLS ⩽ 0.05** (no layout shift)
- **PageSpeed score ⩾ 90** (mobile)

SEO

- Semantic HTML5 (`<header>`, `<nav>`, `<main>`, `<article>`, `<aside>`, `<footer>`)

- All images have `alt` text
- Meta tags per page (title, description, og:image)
- Schema.org markup:
  - `LocalBusiness` + `MedicalBusiness` on homepage
  - `Service` schema on service pages
  - `Person` schema for team members
- Sitemap.xml + robots.txt
- `lang="de-CH"` on `<html>`

Security (Healthcare Context)

- All forms protected with reCAPTCHA v3 or CSRF tokens
- API routes rate-limited (5 req/min for inquiry submission)
- Staff routes protected by middleware (auth check)
- No PII in client-side logs
- All auth events logged server-side (audit trail)
- Session: HttpOnly, Secure, SameSite=Lax cookies
- Auto-logout after 30 min idle on staff dashboard

Data Privacy (Swiss GDPR / DSG)

- Cookie consent banner (Cookiebot or custom)
- Google Fonts self-hosted (no Google CDN)
- Google Analytics: IP anonymization + consent gating
- Contact form data: Supabase EU region
- Privacy policy: `/datenschutz/`
- Clear data handling explanation on form

FILE STRUCTURE (complete scaffold)

```
riviera-med/
├── .env.local
├── .env.example
├── .gitignore
├── next.config.mjs
├── package.json
├── pnpm-lock.yaml
├── tsconfig.json
```

```
├── tailwind.config.ts
├── drizzle.config.ts
│
├── public/
│   ├── fonts/
│   │   ├── switzer-regular.woff2
│   │   ├── switzer-medium.woff2
│   │   └── switzer-semibold.woff2
│   ├── images/
│   │   ├── hero-home.jpg
│   │   ├── team/
│   │   │   ├── muad.jpg
│   │   │   ├── florjan.jpg
│   │   │   └── lenell.jpg
│   │   └── partners/
│   │       ├── spital-thun.svg
│   │       ├── gsi.svg
│   │       └── ...
│   ├── favicon.ico
│   ├── robots.txt
│   └── sitemap.xml
│
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── globals.css
│   │   ├── providers.tsx
│   │   │
│   │   ├── (public)/
│   │   │   ├── page.tsx
│   │   │   ├── leistungen/
│   │   │   │   ├── page.tsx
│   │   │   │   ├── pflege-betreuung/page.tsx
│   │   │   │   ├── hauswirtschaft/page.tsx
│   │   │   │   ├── nachtwachen/page.tsx
│   │   │   │   └── physiotherapie/page.tsx
│   │   │   ├── tarife/page.tsx
│   │   │   ├── ueber-uns/page.tsx
│   │   │   ├── partner/page.tsx
│   │   │   ├── kontakt/page.tsx
│   │   │   ├── datenschutz/page.tsx
│   │   │   └── impressum/page.tsx
│   │   │
│   │   ├── (admin)/
│   │   │   ├── login/page.tsx
│   │   │   ├── forgot/page.tsx
│   │   │   ├── reset/page.tsx
```

```
│   │   │   └── dashboard/
│   │   │       ├── page.tsx
│   │   │       ├── clients/page.tsx
│   │   │       ├── inquiries/page.tsx
│   │   │       ├── schedule/page.tsx
│   │   │       └── reports/page.tsx
│   │   │
│   │   ├── api/
│   │   │   ├── auth/[...nextauth]/route.ts
│   │   │   ├── inquiry/route.ts
│   │   │   ├── pricing/email/route.ts
│   │   │   ├── staff/
│   │   │   │   ├── clients/route.ts
│   │   │   │   ├── inquiries/route.ts
│   │   │   │   └── dashboard/route.ts
│   │   │   └── health/route.ts
│   │   │
│   │   ├── error.tsx
│   │   └── not-found.tsx
│   │
│   ├── components/
│   │   ├── ui/                    ← primitives (shadcn-inspired)
│   │   │   ├── button.tsx
│   │   │   ├── input.tsx
│   │   │   ├── textarea.tsx
│   │   │   ├── select.tsx
│   │   │   ├── checkbox.tsx
│   │   │   ├── radio.tsx
│   │   │   ├── badge.tsx
│   │   │   ├── card.tsx
│   │   │   ├── alert.tsx
│   │   │   ├── accordion.tsx
│   │   │   ├── tabs.tsx
│   │   │   ├── modal.tsx
│   │   │   ├── toast.tsx
│   │   │   └── spinner.tsx
│   │   │
│   │   ├── layout/
│   │   │   ├── SiteHeader.tsx
│   │   │   ├── SiteFooter.tsx
│   │   │   ├── MobileDrawer.tsx
│   │   │   ├── Container.tsx
│   │   │   ├── Section.tsx
│   │   │   └── Grid.tsx
│   │   │
│   │   ├── domain/                ← business components
│   │   │   ├── ServiceCard.tsx
```

```
│   │   │   ├── TariffTable.tsx
│   │   │   ├── TestimonialCard.tsx
│   │   │   ├── TeamCard.tsx
│   │   │   ├── PartnerLogoGrid.tsx
│   │   │   └── EmergencyBanner.tsx
│   │   │
│   │   └── shared/
│   │       ├── PhoneFallback.tsx
│   │       └── TrustStrip.tsx
│   │
│   ├── features/
│   │   ├── inquiry-wizard/
│   │   │   ├── InquiryWizard.tsx
│   │   │   ├── components/
│   │   │   │   ├── WizardShell.tsx
│   │   │   │   ├── ProgressIndicator.tsx
│   │   │   │   ├── StepNavigator.tsx
│   │   │   │   └── ResumeDialog.tsx
│   │   │   ├── steps/
│   │   │   │   ├── Step1Service.tsx
│   │   │   │   ├── Step2Recipient.tsx
│   │   │   │   ├── Step3Urgency.tsx
│   │   │   │   ├── Step4Contact.tsx
│   │   │   │   ├── Step5BestTime.tsx
│   │   │   │   ├── Step6Message.tsx
│   │   │   │   ├── Step7Review.tsx
│   │   │   │   └── Step8Success.tsx
│   │   │   └── hooks/
│   │   │       ├── useInquiryWizard.ts
│   │   │       └── useDraftPersistence.ts
│   │   │
│   │   ├── pricing-calculator/
│   │   │   ├── PricingCalculator.tsx
│   │   │   ├── components/
│   │   │   │   ├── ServiceToggle.tsx
│   │   │   │   ├── DaysSlider.tsx
│   │   │   │   ├── AgeToggle.tsx
│   │   │   │   └── ResultsPanel.tsx
│   │   │   └── lib/
│   │   │       └── calculate.ts
│   │   │
│   │   ├── client-search/
│   │   │   ├── ClientSearchPage.tsx
│   │   │   ├── components/
│   │   │   │   ├── SearchToolbar.tsx
│   │   │   │   ├── FacetSidebar.tsx
│   │   │   │   ├── ResultsTable.tsx
```

```
│   │   │   │   └── Pagination.tsx
│   │   │   └── hooks/
│   │   │       ├── useClientSearch.ts
│   │   │       └── useUrlState.ts
│   │   │
│   │   ├── dashboard/
│   │   │   ├── DashboardPage.tsx
│   │   │   ├── components/
│   │   │   │   ├── DashboardShell.tsx
│   │   │   │   ├── KpiCard.tsx
│   │   │   │   └── widgets/
│   │   │   │       ├── TodaysVisitsWidget.tsx
│   │   │   │       ├── NewInquiriesWidget.tsx
│   │   │   │       ├── ActiveClientsWidget.tsx
│   │   │   │       └── HoursByServiceChart.tsx
│   │   │   └── hooks/
│   │   │       ├── useTodaysVisits.ts
│   │   │       ├── useNewInquiries.ts
│   │   │       └── useAssignInquiry.ts
│   │   │
│   │   └── auth/
│   │       ├── LoginPage.tsx
│   │       ├── ForgotPage.tsx
│   │       ├── ResetPasswordPage.tsx
│   │       ├── components/
│   │       │   ├── LoginForm.tsx
│   │       │   ├── ForgotForm.tsx
│   │       │   ├── ResetForm.tsx
│   │       │   └── IdleWarningBanner.tsx
│   │       └── hooks/
│   │           ├── useLogin.ts
│   │           ├── useIdleTimer.ts
│   │           └── useSessionSync.ts
│   │
│   ├── lib/
│   │   ├── queryClient.ts        ← TanStack Query setup
│   │   ├── db.ts                 ← Drizzle client
│   │   ├── crypto.ts             ← Argon2 password hashing
│   │   ├── email.ts              ← Resend wrapper
│   │   ├── analytics.ts          ← PostHog wrapper
│   │   └── hooks/
│   │       ├── useDebounce.ts
│   │       └── useMediaQuery.ts
│   │
│   ├── server/
│   │   ├── auth.config.ts        ← NextAuth config
│   │   ├── db/
```

```
│   │   │   ├── schema.ts          ← Drizzle schema
│   │   │   └── queries.ts
│   │   └── services/
│   │       ├── inquiry.service.ts
│   │       ├── client.service.ts
│   │       └── auth.service.ts
│   │
│   ├── shared/
│   │   └── schemas/
│   │       ├── inquiry.ts         ← Zod schemas
│   │       ├── client.ts
│   │       └── auth.ts
│   │
│   ├── styles/
│   │   ├── tokens.css             ← Design system CSS variables
│   │   └── fonts.css
│   │
│   └── middleware.ts              ← Route protection
│
├── drizzle/
│   └── migrations/
│
└── tests/
    ├── unit/
    ├── integration/
    └── e2e/
```

IMPLEMENTATION PRIORITY (phased rollout)

Phase 1 — Public Site MVP (Weeks 1-4)

Goal: Launch public-facing site with conversion path.

Deliverables:

- [ ] Homepage (`/`)
- [ ] Services hub (`/leistungen/`)
- [ ] Pricing page with calculator (`/tarife/`)
- [ ] About page (`/ueber-uns/`)
- [ ] Contact page with multi-step form (`/kontakt/`)
- [ ] Footer + legal pages (Datenschutz, Impressum)
- [ ] Email integration (Resend) for inquiry form
- [ ] Design system tokens imported
- [ ] All 37 UI components from design system

- [ ] SEO basics (meta tags, sitemap, schema.org)
- [ ] Cookie consent banner
- [ ] Google Analytics setup

Database schema needed:

- `inquiries` table (id, serviceType, recipient, urgency, name, phone, email, bestTime, message, consent, submittedAt, status)

API endpoints needed:

- `POST /api/inquiry` (form submission)
- `POST /api/pricing/email` (calculator summary email)

No auth yet — staff access to inquiries via direct DB queries initially.

Phase 2 — Staff Authentication (Weeks 5–6)

Goal: Secure staff-only area.

Deliverables:

- [ ] Login page (`/login/`)
- [ ] Forgot password flow (`/forgot/`, `/reset/`)
- [ ] NextAuth v5 setup with Credentials provider
- [ ] Middleware for route protection
- [ ] Password hashing (Argon2id)
- [ ] Rate limiting on login
- [ ] Idle timeout (30 min) with warning banner
- [ ] Cross-tab logout sync

Database schema:

- `users` table (id, email, passwordHash, name, role, createdAt)
- `passwordResetTokens` table (id, userId, token, expiresAt)

API endpoints:

- `POST /api/auth/forgot` (send reset link)
- `POST /api/auth/reset` (verify token, update password)

Phase 3 — Staff Dashboard Basic (Weeks 7–10)

Goal: Internal tools for managing inquiries and clients.

Deliverables:

- [ ] Dashboard home (`/dashboard/`) with 4 KPI cards + 4 widgets
- [ ] New inquiries widget (list + assign)
- [ ] Today's visits widget (read-only initially)
- [ ] Active clients count
- [ ] Weekly hours chart (Recharts)
- [ ] Inquiry detail view (mark as contacted, add notes)
- [ ] Client list (basic CRUD)

Database schema:

- `clients` table
- `visits` table
- `notes` table

API endpoints:

- `GET /api/staff/dashboard` (KPIs + widget data)
- `GET /api/staff/inquiries?status=new`
- `PATCH /api/staff/inquiries/:id` (update status, assignment)
- `GET /api/staff/clients`
- `POST /api/staff/clients` (create from inquiry)

Phase 4 — Advanced Features (Weeks 11–16)

Goal: Faceted search, real-time updates, exports.

Deliverables:

- [ ] Faceted client search (`/dashboard/clients/`)
- [ ] URL-synced filter state
- [ ] Server-side pagination
- [ ] Real-time inquiry notifications (Server-Sent Events)
- [ ] Visit scheduling UI
- [ ] PDF export (Tarifblatt, client summary)
- [ ] Dashboard chart: Hours by service type (monthly)
- [ ] Dashboard chart: Inquiries over time (trend)

CODE PATTERNS & BEST PRACTICES

1. Component Pattern (Compound Component)

tsx

```tsx
// Button with variants
import { cn } from '@/lib/utils'

type ButtonProps = {
  variant?: 'primary' | 'secondary' | 'ghost' | 'destructive'
  size?: 'sm' | 'md' | 'lg'
  disabled?: boolean
  children: React.ReactNode
} & React.ButtonHTMLAttributes<HTMLButtonElement>

export function Button({
  variant = 'primary',
  size = 'md',
  disabled,
  className,
  children,
  ...props
}: ButtonProps) {
  return (
    <button
      className={cn(
        'inline-flex items-center justify-center gap-2 rounded-sm font-medium transi
        'focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outlin
        'disabled:opacity-40 disabled:cursor-not-allowed disabled:pointer-events-non
        {
          'bg-primary-700 text-neutral-50 hover:bg-primary-800 active:bg-primary-900
          'border border-neutral-300 text-neutral-800 hover:bg-neutral-100': variant
          'text-neutral-700 hover:bg-neutral-100': variant === 'ghost',
          'bg-error-500 text-neutral-50 hover:bg-error-700': variant === 'destructiv
        },
        {
          'h-8 px-3 text-sm': size === 'sm',
          'h-10 px-4 text-base': size === 'md',
          'h-12 px-6 text-lg': size === 'lg',
        },
        className
      )}
      disabled={disabled}
      {...props}
    >
      {children}
    </button>
  )
}
```

2. Form Field Pattern (React Hook Form + Zod)

tsx

```tsx
// Field wrapper component
export function Field({
  label,
  required,
  error,
  helper,
  children,
}: {
  label: string
  required?: boolean
  error?: string
  helper?: string
  children: React.ReactNode
}) {
  return (
    <div className="space-y-2">
      <label className="block text-sm font-medium text-neutral-800">
        {label} {required && <span className="text-error-500">*</span>}
      </label>
      {children}
      {helper && !error && (
        <p className="text-sm text-neutral-500">{helper}</p>
      )}
      {error && (
        <p className="text-sm text-error-700" role="alert">
          {error}
        </p>
      )}
    </div>
  )
}

// Usage in form
<Field label="Name" required error={errors.name?.message}>
  <input
    type="text"
    {...register('name')}
    className="w-full h-12 px-4 border border-neutral-300 rounded-sm"
  />
</Field>
```

3. TanStack Query Pattern (with optimistic updates)

tsx

```tsx
// Mutation with optimistic update
import { useMutation, useQueryClient } from '@tanstack/react-query'

export function useAssignInquiry() {
  const qc = useQueryClient()

  return useMutation({
    mutationFn: ({ inquiryId, staffId }: { inquiryId: string, staffId: string }) =>
      fetch(`/api/staff/inquiries/${inquiryId}/assign`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ staffId }),
      }).then(r => {
        if (!r.ok) throw new Error('Assignment failed')
        return r.json()
      }),

    onMutate: async ({ inquiryId, staffId }) => {
      await qc.cancelQueries({ queryKey: ['inquiries', 'new'] })
      const previous = qc.getQueryData(['inquiries', 'new'])

      qc.setQueryData(['inquiries', 'new'], (old: Inquiry[]) =>
        old.map(i => i.id === inquiryId ? { ...i, assignedTo: staffId } : i)
      )

      return { previous }
    },

    onError: (err, vars, context) => {
      qc.setQueryData(['inquiries', 'new'], context?.previous)
      toast.error('Konnte nicht zugewiesen werden.')
    },

    onSettled: () => {
      qc.invalidateQueries({ queryKey: ['inquiries'] })
    },
  })
}
```

4. Server Action Pattern (Next.js App Router)

tsx

```ts
// app/api/inquiry/route.ts
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'
import { inquirySchema } from '@/shared/schemas/inquiry'
import { db } from '@/lib/db'
import { inquiries } from '@/server/db/schema'
import { sendEmail } from '@/lib/email'

export async function POST(req: NextRequest) {
  try {
    const body = await req.json()
    const data = inquirySchema.parse(body)

    // Insert into DB
    const [inquiry] = await db.insert(inquiries).values({
      ...data,
      status: 'new',
      submittedAt: new Date(),
    }).returning()

    // Send email to staff
    await sendEmail({
      to: 'info@riviera-med.com',
      subject: `Neue Anfrage: ${data.name}`,
      html: `<p>Neue Anfrage von ${data.name}</p>...`,
    })

    // Send auto-reply to user
    if (data.email) {
      await sendEmail({
        to: data.email,
        subject: 'Ihre Anfrage bei Riviera Med',
        html: `<p>Guten Tag ${data.name},</p>...`,
      })
    }

    return NextResponse.json({ success: true, id: inquiry.id }, { status: 201 })
  } catch (error) {
    if (error instanceof z.ZodError) {
      return NextResponse.json({ error: error.errors }, { status: 400 })
    }
    console.error('Inquiry submission error:', error)
    return NextResponse.json({ error: 'Internal error' }, { status: 500 })
  }
}
```

SUCCESS CRITERIA (definition of done)

Functional

- [ ] All 6 public pages render correctly
- [ ] Inquiry form submits successfully, sends emails
- [ ] Pricing calculator computes correct totals
- [ ] Staff can login, view dashboard, manage inquiries
- [ ] All links work (no 404s)
- [ ] Forms validate properly (Zod schemas)
- [ ] Responsive on mobile (375px), tablet (768px), desktop (1280px+)

Performance

- [ ] Lighthouse score ⩾ 90 (Performance, Accessibility, Best Practices, SEO)
- [ ] LCP ⩽ 1.5s on homepage
- [ ] No CLS (layout shifts)
- [ ] No console errors in production

Accessibility

- [ ] All interactive elements ⩾ 48px touch target
- [ ] All images have alt text
- [ ] Form fields have labels
- [ ] Focus indicators visible on keyboard nav
- [ ] Works with screen reader (NVDA/VoiceOver tested)
- [ ] Honors prefers-reduced-motion
- [ ] Color contrast ⩾ 4.5:1 for all text

Security

- [ ] All API routes rate-limited
- [ ] Staff routes protected by middleware
- [ ] Passwords hashed with Argon2id
- [ ] CSRF protection enabled
- [ ] No PII in logs
- [ ] Environment variables not committed

SEO

- [ ] Meta tags on all pages
- [ ] Schema.org markup on homepage, services
- [ ] Sitemap.xml generated
- [ ] robots.txt configured

- [ ] Google Search Console submitted

Content

- [ ] All copy in German (de-CH)
- [ ] No Lorem ipsum
- [ ] Phone number `079 237 55 21` correct everywhere
- [ ] Email `info@riviera-med.com` correct
- [ ] Team photos uploaded
- [ ] Partner logos uploaded

DEPLOYMENT CHECKLIST

Pre-launch

- [ ] Environment variables set in Vercel
- [ ] Domain `riviera-med.ch` pointed to Vercel
- [ ] SSL certificate active
- [ ] Database migrations run
- [ ] Seed data loaded (1 staff user for login)
- [ ] Resend domain verified (for emails from `info@riviera-med.com`)
- [ ] PostHog project created
- [ ] Sentry project created
- [ ] Google Analytics property created
- [ ] Cookie consent banner configured

Launch Day

- [ ] Deploy to production
- [ ] Test inquiry form submission end-to-end
- [ ] Test staff login
- [ ] Submit sitemap to Google Search Console
- [ ] Monitor error logs (Sentry)
- [ ] Monitor analytics (PostHog)

Post-Launch (Week 1)

- [ ] Review inquiry conversion rate
- [ ] Check form abandonment funnel
- [ ] Fix any browser-specific bugs
- [ ] Optimize images further if needed
- [ ] Set up weekly automated reports

CONTACT & SUPPORT

Client: Riviera Med GmbH
Email: info@riviera-med.com
Phone: 079 237 55 21
Address: Scheibenstrasse 3, 3600 Thun, Switzerland

Project artifacts:

- Design system tokens: `tokens.json`, `tokens.css`
- Website copy: `riviera-med-website-copy.md`
- Technical blueprint: `riviera-med-blueprint.md`
- Interactive modules architecture: `riviera-med-modules-architecture.md`

End of master build specification.
This document is your complete source of truth. Paste it into your AI coding assistant and start building.